

Question 24 – Distributed Database Query Processor

Aim

Solve $T(n)=5T(n/2)+n$ using the Master Theorem. Identify parameters, justify the case, and derive the asymptotic complexity.

Algorithm

1. Read n.
2. Apply the recurrence recursively.
3. Compute additional work.
4. Print the estimated cost.

Source Code

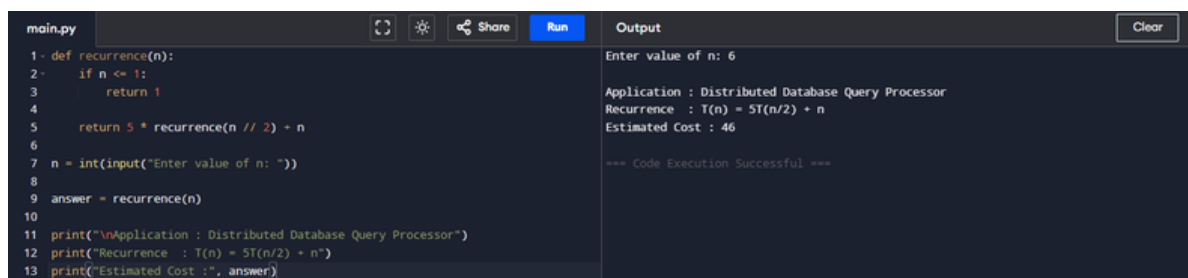
```
def recurrence(n):
    if n <= 1:
        return 1

    return 5 * recurrence(n // 2) + n

n = int(input("Enter value of n: "))

answer = recurrence(n)

print("\nApplication : Distributed Database Query Processor")
print("Recurrence :  $T(n) = 5T(n/2) + n$ ")
print("Estimated Cost :", answer)
```



The screenshot shows a code editor with a file named 'main.py'. The code defines a recursive function 'recurrence(n)' that calculates the cost of a distributed database query processor. The function has a base case where n is less than or equal to 1, returning 1. Otherwise, it returns 5 times the cost of a recursive call with n divided by 2, plus n. The main part of the code prompts the user to enter a value for n, which is then passed to the 'recurrence' function. The output of the program is displayed in a separate window, showing the input '6', the application name, the recurrence formula, and the estimated cost of 46. A message at the bottom indicates that the code execution was successful.

```
main.py  [Icons]  Run  Output  Clear
1- def recurrence(n):
2-     if n <= 1:
3-         return 1
4-
5-     return 5 * recurrence(n // 2) + n
6-
7- n = int(input("Enter value of n: "))
8-
9- answer = recurrence(n)
10-
11- print("\nApplication : Distributed Database Query Processor")
12- print("Recurrence :  $T(n) = 5T(n/2) + n$ ")
13- print("Estimated Cost :", answer)
```

Enter value of n: 6

Application : Distributed Database Query Processor
Recurrence : $T(n) = 5T(n/2) + n$
Estimated Cost : 46

=== Code Execution Successful ===

Analysis

Time Complexity: $\Theta(n^{\log_2 5}) \approx \Theta(n^{2.322})$

Derived using Master Theorem (Case 1).

Space Complexity: $O(\log n)$

Due to the recursive call stack.

Conclusion

The implementation demonstrates the recurrence and supports the theoretical complexity analysis.